

IRTRA SAFELIGHT

Sistema autónomo de iluminación de señalización y alerta de seguridad

DOCUMENTO FORMAL DE INVESTIGACIÓN

Proyecto Final de Tecnología Aplicada – Undécimo

Parque Mundo Petapa – IRTRA

Integrantes:

Luisa Valladares

Daniela González

Ana Laura

Grado y sección: 11mo D

Docente: Eswin Godoy

Fecha de entrega: 10 de septiembre de 2026

ÍNDICE DE CONTENIDO

Contenido

1. Introducción	5
2. Planteamiento del problema.....	5
3. Justificación	5
4. Objetivos	6
4.1 Objetivo general.....	6
4.2 Objetivos específicos.....	6
5. TEMA 1. Contexto de Mundo Petapa y necesidad de seguridad	8
5.1 Mundo Petapa como contexto del proyecto.....	8
5.2 Seguridad preventiva.....	8
5.3 Señalización por proximidad.....	8
5.4 Alerta autónoma	8
6. TEMA 2. Arduino Uno R3 y automatización	8
6.1 Microcontrolador Arduino Uno R3.....	8
6.2 Entradas del sistema.....	8
6.3 Salidas del sistema	8
6.4 Ciclo automático de control	9
7. TEMA 3. Sensor ultrasónico HC-SR04 y medición de distancia.....	9
7.1 Principio de medición ultrasónica.....	9
7.2 Función medirDistancia()	9
7.3 Umbrales de 80 cm y 200 cm.....	9
7.4 Prioridad de la alarma sobre la distancia.....	9
8. TEMA 4. Sensor PIR, botón, LED RGB y buzzer	9
8.1 Sensor PIR	9
8.2 Detección por flanco de subida	9
8.3 Botón de restablecimiento	10
8.4 LED RGB y buzzer.....	10
9. TEMA 5. Programación, lógica de seguridad y ODS.....	10
9.1 Variables y memoria de estado.....	10
9.2 Funciones reutilizables	10
9.3 ODS relacionados.....	10
9.4 Mejoras futuras.....	10
10. Diseño técnico del sistema	12
10.1 Lista de materiales.....	12

10.2 Asignación exacta de pines.....	12
10.3 Lógica de operación	12
10.4 Diagrama lógico textual.....	12
11. Código C++ utilizado en Tinkercad	13
12. Matriz de funcionamiento y pruebas.....	15
EVIDENCIA DEL CIRCUITO EN TINKERCAD	16
13. Resultados esperados y validación	17
14. Alcances y limitaciones.....	17
15. Conclusiones	17
16. Referencias bibliográficas	19
Anexo A. Evidencias y enlaces de entrega	19
Anexo B. Lista de comprobación final.....	19

SECCIÓN I

Planteamiento general del proyecto

1. Introducción

El presente documento desarrolla IRTRA SafeLight, una propuesta académica de automatización para el Parque Mundo Petapa del IRTRA. El sistema combina señalización luminosa por proximidad y una alerta de seguridad activada por movimiento. Su núcleo de control es un Arduino Uno R3 programado en C++ y simulado en Tinkercad Circuits. La solución utiliza un sensor ultrasónico HC-SR04 para medir distancia, un sensor PIR para detectar movimiento, un LED RGB para representar estados de proximidad, un buzzer como alarma sonora y un botón para desactivar la alarma.

Mundo Petapa fue inaugurado el 26 de marzo de 1976, se ubica en la Avenida Petapa y 42 calle de la zona 12 de la Ciudad de Guatemala, ocupa 11.25 hectáreas y tiene capacidad para 12,500 visitantes. El parque ofrece atracciones y juegos electromecánicos, por lo que constituye un contexto apropiado para plantear una solución educativa de automatización y señalización preventiva (IRTRA, s. f.).

La guía del proyecto solicita identificar una problemática operacional, de seguridad, entretenimiento o energía y diseñar un sistema electrónico autónomo que la resuelva. Asimismo, exige Arduino Uno R3, simulación en Tinkercad, programación exclusivamente en C++ y el uso de al menos cuatro componentes. IRTRA SafeLight cumple esos criterios mediante una lógica automática que interpreta distancia y movimiento para producir respuestas visuales y sonoras.

2. Planteamiento del problema

En un parque de diversiones existen zonas de circulación, accesos o espacios técnicos en los que es útil advertir la proximidad de una persona u objeto y detectar movimiento que pueda requerir atención. Para este proyecto se plantea una situación hipotética: un área de Mundo Petapa necesita una señalización automática que indique, mediante colores, qué tan cerca se encuentra una persona u objeto y que, además, active una alarma cuando el sensor PIR detecte un nuevo movimiento.

La problemática es académica y no constituye una afirmación de que Mundo Petapa presente actualmente una deficiencia de seguridad. La propuesta busca demostrar cómo sensores, actuadores y programación pueden integrarse para producir una respuesta automática y comprensible para el usuario.

Pregunta de investigación: ¿Cómo puede un sistema basado en Arduino Uno R3 utilizar medición ultrasónica de distancia y detección PIR de movimiento para generar señalización luminosa por niveles de proximidad y una alarma de seguridad que pueda ser restablecida manualmente?

3. Justificación

IRTRA SafeLight permite aplicar conocimientos de electrónica, robótica y programación a un problema de seguridad preventiva. El sensor ultrasónico proporciona una referencia de distancia; el LED RGB transforma esa información en una señal visual fácil de interpretar; el PIR detecta el inicio de un movimiento; el buzzer genera una alerta audible; y el botón ofrece control manual para restablecer la alarma. La combinación de estos elementos crea un sistema autónomo que responde al entorno sin requerir que una persona controle cada salida.

Arduino Uno R3 está basado en el ATmega328P y dispone de 14 pines digitales de entrada/salida, seis de ellos utilizables como PWM, además de seis entradas analógicas. Estas características permiten conectar los sensores y actuadores empleados en el prototipo (Arduino, s. f.).

La propuesta se vincula principalmente con el ODS 11, Ciudades y comunidades sostenibles, por su enfoque en espacios más seguros, y de manera complementaria con el ODS 9, Industria, innovación e infraestructura, por la aplicación de automatización. También puede relacionarse con el ODS 7 cuando la señalización

automatizada se entiende como parte de un uso controlado de dispositivos eléctricos; sin embargo, el código actual no mide consumo energético ni utiliza un sensor de luz, por lo que no se atribuye un ahorro energético cuantificado.

4. Objetivos

4.1 Objetivo general

Diseñar y simular un sistema autónomo de señalización luminosa y alerta de seguridad para un escenario hipotético de Mundo Petapa, utilizando Arduino Uno R3, un sensor ultrasónico HC-SR04, un sensor PIR, un LED RGB, un buzzer y un botón, programados en C++ dentro de Tinkercad Circuits.

4.2 Objetivos específicos

- Medir la distancia de una persona u objeto mediante el sensor ultrasónico HC-SR04.
- Representar tres niveles de proximidad mediante los colores verde, amarillo y rojo del LED RGB.
- Detectar el inicio de movimiento mediante un sensor PIR y activar una alarma visual y sonora.
- Permitir que la alarma sea desactivada manualmente mediante un botón configurado con INPUT_PULLUP.
- Programar y probar toda la lógica en C++ dentro de Tinkercad Circuits.
- Documentar el funcionamiento, las conexiones, las pruebas, las limitaciones y la relación del proyecto con los ODS.

SECCIÓN II

Fundamentos: 5 temas principales y 20 subtemas

5. TEMA 1. Contexto de Mundo Petapa y necesidad de seguridad

5.1 Mundo Petapa como contexto del proyecto

Mundo Petapa es uno de los parques del Instituto de Recreación de los Trabajadores de la Empresa Privada de Guatemala. La información institucional señala que se encuentra en la zona 12 de la Ciudad de Guatemala y cuenta con diversas atracciones recreativas y juegos electromecánicos (IRTRA, s. f.). Para este trabajo, el parque funciona como escenario de aplicación de una solución tecnológica de carácter educativo.

5.2 Seguridad preventiva

La seguridad preventiva busca anticipar situaciones que podrían representar un riesgo. En IRTRA SafeLight, la prevención se representa mediante dos mecanismos: una señal visual de proximidad y una alarma activada por movimiento. El objetivo no es sustituir al personal de seguridad, sino demostrar cómo un sistema electrónico puede apoyar la detección y señalización.

5.3 Señalización por proximidad

La señalización por colores permite comunicar rápidamente un estado. El código utiliza verde cuando la distancia es igual o mayor a 200 cm, amarillo cuando es menor de 200 cm pero no menor de 80 cm, y rojo cuando la distancia es menor de 80 cm. Estos límites son decisiones de diseño del equipo para la simulación y no corresponden a una norma oficial del IRTRA.

5.4 Alarma autónoma

La alarma se activa cuando el sensor PIR cambia de LOW a HIGH, es decir, cuando el programa detecta el inicio de un evento de movimiento. Una vez activada, permanece encendida aunque el PIR deje de detectar movimiento, hasta que el usuario presiona el botón. Esta memoria de estado se logra mediante la variable booleana `alarmaActiva`.

6. TEMA 2. Arduino Uno R3 y automatización

6.1 Microcontrolador Arduino Uno R3

Arduino Uno R3 es una placa basada en el microcontrolador ATmega328P. Integra entradas y salidas digitales, entradas analógicas y recursos suficientes para proyectos educativos de control. En este prototipo actúa como centro de decisión: recibe señales de los sensores, ejecuta las condiciones del programa y controla el RGB y el buzzer (Arduino, s. f.).

6.2 Entradas del sistema

Las entradas son el sensor ultrasónico, el sensor PIR y el botón. El HC-SR04 utiliza un pin de disparo y uno de eco; el PIR entrega un estado digital; y el botón se configura como `INPUT_PULLUP`, por lo que normalmente se lee HIGH y al presionarlo se lee LOW.

6.3 Salidas del sistema

Las salidas son los tres canales del LED RGB y el buzzer. Los pines rojo, verde y azul se controlan mediante `digitalWrite`. La función `setColor(r, g, b)` agrupa el control de los tres canales y simplifica la selección de rojo, amarillo o verde. El buzzer se activa únicamente cuando `alarmaActiva` es verdadera.

6.4 Ciclo automático de control

La función `loop()` se ejecuta repetidamente. En cada ciclo se mide la distancia, se lee el PIR y se lee el botón. Luego se actualiza el estado de alarma y finalmente se selecciona la salida correspondiente. Al final se imprimen datos en el monitor serial y se espera 100 milisegundos antes de iniciar el siguiente ciclo.

7. TEMA 3. Sensor ultrasónico HC-SR04 y medición de distancia

7.1 Principio de medición ultrasónica

El HC-SR04 calcula distancia a partir del tiempo que tarda un pulso ultrasónico en viajar hasta un objeto y regresar. Su documentación indica que el módulo se dispara con un pulso de aproximadamente 10 microsegundos y que la distancia se obtiene a partir del tiempo del eco y la velocidad del sonido, considerando el recorrido de ida y vuelta (HC-SR04, s. f.).

7.2 Función `medirDistancia()`

El código primero mantiene `trigPin` en LOW durante 2 microsegundos, luego lo coloca en HIGH durante 10 microsegundos y finalmente vuelve a LOW. Después `pulseIn(echoPin, HIGH)` mide la duración del pulso de eco. La expresión `duracion * 0.034 / 2` convierte ese tiempo en una aproximación de distancia en centímetros.

7.3 Umbrales de 80 cm y 200 cm

La lógica establece tres zonas. Menos de 80 cm corresponde a proximidad alta y se representa en rojo. Desde 80 cm hasta menos de 200 cm corresponde a proximidad intermedia y se representa en amarillo. A partir de 200 cm se considera una distancia más segura y se representa en verde. Estos umbrales pueden ajustarse si las pruebas del prototipo muestran la necesidad de hacerlo.

7.4 Prioridad de la alarma sobre la distancia

Cuando `alarmaActiva` es verdadera, el programa no evalúa los colores de distancia: inmediatamente coloca el RGB en rojo y enciende el buzzer. Esto significa que el estado de alarma tiene prioridad sobre la señalización normal de proximidad. Al desactivar la alarma, el sistema vuelve a mostrar el color correspondiente a la distancia medida.

8. TEMA 4. Sensor PIR, botón, LED RGB y buzzer

8.1 Sensor PIR

Los sensores PIR se utilizan para detectar cambios de radiación infrarroja asociados al movimiento de cuerpos dentro de su campo de detección. En aplicaciones con Arduino, su salida se maneja como una señal digital HIGH o LOW. El código de este proyecto compara la lectura actual con la anterior para identificar específicamente la transición de LOW a HIGH (Adafruit, 2024).

8.2 Detección por flanco de subida

La condición `movimiento == HIGH && pirAnterior == LOW` detecta únicamente el momento en que inicia una detección. Después, `pirAnterior` se actualiza con el valor actual. Esta estrategia evita ejecutar repetidamente la activación durante todos los ciclos en los que el PIR permanezca en HIGH. Para que vuelva a existir un nuevo evento, el PIR debe regresar a LOW y posteriormente pasar otra vez a HIGH.

8.3 Botón de restablecimiento

El botón está conectado al pin A4 y se configura con INPUT_PULLUP. Por ello, al presionarlo la lectura es LOW. La instrucción `if (boton == LOW) alarmaActiva = false;` otorga al botón prioridad para apagar la alarma. Si el PIR continúa en HIGH después de presionarlo, la alarma no vuelve a activarse inmediatamente porque no existe un nuevo cambio de LOW a HIGH.

8.4 LED RGB y buzzer

El RGB comunica visualmente el estado del sistema. Rojo representa alarma o proximidad menor de 80 cm; amarillo representa distancia intermedia; y verde representa una distancia igual o mayor de 200 cm. El buzzer se mantiene apagado durante la señalización normal de distancia y se enciende solamente cuando la alarma está activa.

9. TEMA 5. Programación, lógica de seguridad y ODS

9.1 Variables y memoria de estado

El programa utiliza constantes para identificar los pines y variables para conservar información. `alarmaActiva` almacena si la alarma está encendida o apagada; `pirAnterior` conserva la lectura previa del PIR. Esta última variable permite comparar el estado anterior con el actual y detectar el inicio del movimiento.

9.2 Funciones reutilizables

La función `setColor()` centraliza el control del LED RGB y evita repetir tres instrucciones cada vez que se desea cambiar de color. La función `medirDistancia()` encapsula el procedimiento de disparo, lectura del eco y cálculo de centímetros. Dividir el programa en funciones mejora su organización y facilita su explicación.

9.3 ODS relacionados

El proyecto se relaciona principalmente con el ODS 11 porque propone una herramienta de señalización y alerta orientada a un espacio de uso público. También se relaciona con el ODS 9 por aplicar tecnología e innovación a una necesidad operativa. Como relación complementaria, el ODS 7 incluye la meta 7.3 de mejorar la eficiencia energética; sin embargo, el prototipo actual no mide consumo ni controla iluminación ambiental, por lo que cualquier aporte energético debe presentarse únicamente como una posibilidad de ampliación futura (Naciones Unidas, s. f.).

9.4 Mejoras futuras

Una versión futura podría incorporar una fotorresistencia LDR para que la señalización luminosa se habilite únicamente cuando exista poca luz, temporización para apagar salidas después de cierto tiempo, varios sensores para cubrir áreas más amplias, registro de eventos, comunicación inalámbrica o integración con un centro de monitoreo. Estas mejoras no forman parte del código entregado actualment

SECCIÓN III

Diseño técnico y funcionamiento real del código

10. Diseño técnico del sistema

10.1 Lista de materiales

Componente	Cantidad	Pin(es) según código	Función
Arduino Uno R3	1	—	Procesamiento y control
Sensor ultrasónico HC-SR04	1	Trig A2 / Echo A1	Medir distancia
Sensor PIR	1	A5	Detectar inicio de movimiento
LED RGB	1	R=5 / G=6 / B=9	Señalización verde, amarilla y roja
Buzzer	1	8	Alerta sonora
Push button	1	A4	Desactivar/restablecer alarma
Resistencias	Según montaje	—	Protección/limitación de corriente
Protoboard y cables	1 juego	—	Interconexión del circuito

10.2 Asignación exacta de pines

- A2: Trigger del HC-SR04.
- A1: Echo del HC-SR04.
- A5: salida del sensor PIR.
- Pin 5: canal rojo del LED RGB.
- Pin 6: canal verde del LED RGB.
- Pin 9: canal azul del LED RGB.
- Pin 8: buzzer.
- A4: botón configurado con INPUT_PULLUP.

10.3 Lógica de operación

1. El Arduino mide continuamente la distancia con el HC-SR04.
2. Lee el estado actual del PIR y del botón.
3. Si el PIR cambia de LOW a HIGH, la variable alarmaActiva pasa a true.
4. Si el botón se presiona, alarmaActiva pasa a false.
5. Si la alarma está activa, el RGB se pone rojo y el buzzer se enciende.
6. Si no hay alarma y la distancia es menor de 80 cm, el RGB se pone rojo sin buzzer.
7. Si no hay alarma y la distancia es menor de 200 cm, el RGB se pone amarillo sin buzzer.
8. Si no hay alarma y la distancia es igual o mayor de 200 cm, el RGB se pone verde sin buzzer.
9. El monitor serial muestra distancia y estado del botón.
10. El ciclo se repite cada 100 ms aproximadamente, además del tiempo empleado por la medición.

10.4 Diagrama lógico textual

INICIO → configurar pines y comunicación serial → medir distancia → leer PIR → leer botón → ¿PIR pasó de LOW a HIGH? Sí: activar alarma → actualizar estado anterior del PIR → ¿botón presionado? Sí: apagar alarma →

¿alarma activa? Sí: RGB rojo + buzzer encendido; No: evaluar distancia → <80 cm: rojo; <200 cm: amarillo; ≥200 cm: verde → imprimir datos seriales → esperar 100 ms → repetir.

11. Código C++ utilizado en Tinkercad

El siguiente código corresponde al programa proporcionado por el equipo y constituye la base real del análisis técnico de este documento.

```
const int trigPin = A2;
const int echoPin = A1;
const int pirPin = A5;
const int redPin = 5;
const int greenPin = 6;
const int bluePin = 9;
const int buzzerPin = 8;
const int buttonPin = A4;

bool alarmaActiva = false;
int pirAnterior = LOW;

void setColor(int r, int g, int b) {
    digitalWrite(redPin, r);
    digitalWrite(greenPin, g);
    digitalWrite(bluePin, b);
}

long medirDistancia() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duracion = pulseIn(echoPin, HIGH);
    return duracion * 0.034 / 2;
}

void setup() {
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    pinMode(pirPin, INPUT);
    pinMode(redPin, OUTPUT);
    pinMode(greenPin, OUTPUT);
    pinMode(bluePin, OUTPUT);
    pinMode(buzzerPin, OUTPUT);
    pinMode(buttonPin, INPUT_PULLUP);
    Serial.begin(9600);
}

void loop() {
    long distancia = medirDistancia();
    int movimiento = digitalRead(pirPin);
    int boton = digitalRead(buttonPin);

    // Solo activa la alarma en el momento en que el PIR pasa de LOW a HIGH
    if (movimiento == HIGH && pirAnterior == LOW) {
        alarmaActiva = true;
    }
    pirAnterior = movimiento;

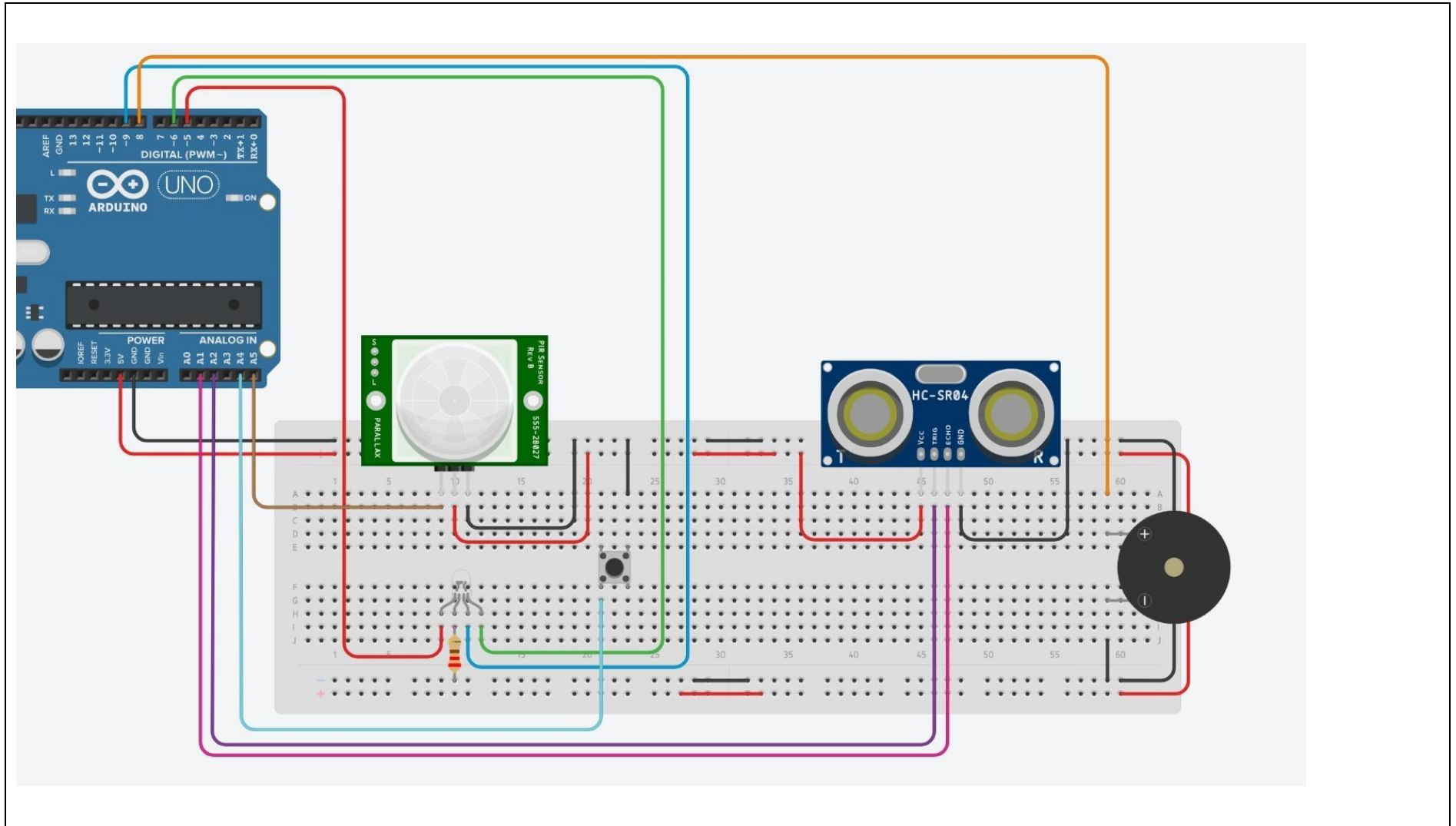
    // El botón tiene la última palabra: siempre puede apagar la alarma
    if (boton == LOW) {
        alarmaActiva = false;
    }
}
```

```
if (alarmaActiva) {  
    setColor(HIGH, LOW, LOW);  
    digitalWrite(buzzerPin, HIGH);  
} else if (distancia < 80) {  
    setColor(HIGH, LOW, LOW);  
    digitalWrite(buzzerPin, LOW);  
} else if (distancia < 200) {  
    setColor(HIGH, HIGH, LOW);  
    digitalWrite(buzzerPin, LOW);  
} else {  
    setColor(LOW, HIGH, LOW);  
    digitalWrite(buzzerPin, LOW);  
}  
  
Serial.print("Distancia: ");  
Serial.print(distancia);  
Serial.print(" | Boton: ");  
Serial.println(boton);  
  
delay(100);  
}
```

12. Matriz de funcionamiento y pruebas

Caso	PIR	Botón	Distancia	RGB	Buzzer	Resultado esperado
1	LOW	HIGH	≥ 200 cm	Verde	OFF	Estado normal; distancia lejana
2	LOW	HIGH	80–199 cm	Amarillo	OFF	Advertencia de proximidad media
3	LOW	HIGH	< 80 cm	Rojo	OFF	Advertencia de proximidad alta
4	LOW→HIGH	HIGH	Cualquiera	Rojo	ON	Se activa y queda memorizada la alarma
5	HIGH	LOW (presionado)	Cualquiera	Según distancia	OFF	El botón desactiva la alarma
6	HIGH después del reset	HIGH	Cualquiera	Según distancia	OFF	No se reactiva hasta que PIR vuelva LOW y luego HIGH
7	LOW→HIGH tras nuevo evento	HIGH	Cualquiera	Rojo	ON	Nuevo movimiento reactiva la alarma

EVIDENCIA DEL CIRCUITO EN TINKERCAD



13. Resultados esperados y validación

El sistema se considera correctamente implementado si el LED RGB cambia de verde a amarillo y rojo conforme disminuye la distancia, si un nuevo evento del PIR activa la alarma roja y sonora, y si el botón logra desactivarla. También debe comprobarse que, después de presionar el botón mientras el PIR continúa en HIGH, la alarma permanezca apagada hasta que ocurra un nuevo ciclo LOW→HIGH del sensor PIR.

El monitor serial debe mostrar la distancia calculada y el valor del botón. Esta salida sirve como apoyo para verificar que las lecturas del circuito coincidan con la respuesta visual y sonora observada.

14. Alcances y limitaciones

El proyecto es una simulación educativa. Los umbrales de 80 cm y 200 cm son parámetros definidos por el equipo y no deben interpretarse como distancias oficiales de seguridad del IRTRA. El prototipo tampoco sustituye sistemas profesionales de vigilancia, evacuación, control de acceso o detección certificados.

El código no incorpora una fotorresistencia LDR ni mide iluminación ambiental. Por esta razón, la expresión “iluminación” en el nombre del proyecto se refiere a señalización luminosa mediante LED RGB. Si se desea que el sistema encienda luminarias únicamente durante la noche, sería necesario añadir un LDR y modificar el código.

La función pulseIn() no incluye un tiempo máximo explícito. En una implementación física avanzada sería recomendable considerar manejo de errores o timeout si no se recibe eco. Para la entrega académica, debe verificarse que la simulación funcione de forma estable con la configuración de Tinkercad.

15. Conclusiones

- IRTRA SafeLight integra correctamente medición de distancia, detección de movimiento, señalización RGB, alarma sonora y restablecimiento manual en una sola lógica de control.
- El sensor HC-SR04 permite clasificar la proximidad en tres rangos definidos por el equipo: verde para 200 cm o más, amarillo para 80 a 199 cm y rojo para menos de 80 cm.
- El sensor PIR no activa la alarma de forma continua por nivel; el código detecta el cambio de LOW a HIGH y memoriza la alarma mediante la variable alarmaActiva.
- El botón configurado con INPUT_PULLUP permite apagar la alarma y tiene prioridad dentro de cada ciclo del programa.
- La alarma tiene prioridad sobre la señalización de distancia: mientras está activa, el RGB permanece rojo y el buzzer encendido.
- El proyecto cumple con el enfoque de automatización solicitado por la guía y se relaciona con seguridad e innovación; cualquier afirmación de ahorro energético requeriría componentes y mediciones adicionales.

SECCIÓN IV

Referencias bibliográficas y anexos

16. Referencias bibliográficas

Arduino. (s. f.). Arduino UNO R3. Arduino Documentation. <https://docs.arduino.cc/hardware/uno-rev3>

Adafruit. (2024). PIR Motion Sensor: Using a PIR w/Arduino. Adafruit Learning System.

<https://learn.adafruit.com/pir-passive-infrared-proximity-motion-sensor/using-a-pir-w-arduino>

HC-SR04. (s. f.). Ultrasonic ranging module HC-SR04 [Ficha técnica].

<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>

Instituto de Recreación de los Trabajadores de la Empresa Privada de Guatemala (IRTRA). (s. f.). Parque Mundo Petapa. <https://irtra.org.gt/parques/?pid=59>

Naciones Unidas. (s. f.). Objetivo 7: Energía asequible y no contaminante. <https://sdgs.un.org/goals/goal7>

Naciones Unidas. (s. f.). Objetivos de Desarrollo Sostenible. <https://sdgs.un.org/es/goals>

Anexo A. Evidencias y enlaces de entrega

Enlace compartido de Tinkercad: https://www.tinkercad.com/things/d3m8BDxgXda-proyecto-grupal?sharecode=BIL5towh-f7JRXO21zoofxQ7O7E-fNYQNoFX6cDgH_Q

Estado final del circuito: ☐ Funciona al 100% ☐ Requiere correcciones

Captura de Tinkercad: insertar en la página horizontal de la matriz de pruebas.

Enlace del repositorio GitHub: _____

Nombre oficial del repositorio: ProyectoFinalTICS_[Apellido1]_[Apellido2]_[Apellido3]_11D_IRTRASafeLight

Enlace de GitHub Pages: _____

Enlace del podcast (máximo 5 min): <https://we.tl/t-1HyZnFeYEjdn9Pp7>

Archivo de infografía / enlace: _____ ARCHIVO EN IMAGEN (HITHUB) _____

Anexo B. Lista de comprobación final

☐ Circuito en Tinkercad con Arduino Uno R3 y mínimo cuatro componentes.

☐ Código exclusivamente en C++.

☐ Los tres integrantes tienen acceso de edición al circuito.

☐ Circuito probado y funcional al 100%.

☐ Documento Word y PDF.

☐ Hojas de colchón entre secciones principales.

☐ Al menos una página horizontal.

☐ Encabezados, pies y numeración de página.

☐ 5 temas principales y al menos 18 subtemas (este documento contiene 20).

☐ Citas parentéticas y bibliografía APA generada con la herramienta nativa de Word.

☐ Infografía con nombre de solución, integrantes, avatares, resumen, justificación, ODS, materiales, estado, captura Tinkercad, GitHub y podcast.

☐ Podcast de máximo 5 minutos.

☐ Landing page en GitHub Pages con archivo principal index.html.

☐ Repositorio con nomenclatura exacta solicitada por la guía.